

$$\begin{aligned}
\mathcal{I}_1^+ &= \mathcal{I}_1^- + H^T R^{-1} H \\
&= 0.7183 \\
K_1 &= (\mathcal{I}_1^+)^{-1} H^T R^{-1} \\
&= \begin{bmatrix} 0.6961 & 0.2785 & 0.0006 \end{bmatrix} \\
\hat{x}_1^- &= F \hat{x}_0^+ \\
&= 0.95 \\
\hat{x}_1^+ &= \hat{x}_1^- + K_1 (y_1 - H \hat{x}_1^-) \\
&= 5.1922
\end{aligned} \tag{6.35}$$

The information filter requires the inversion of Q and R , but in many applications these matrices are constant and can therefore be inverted offline. The only other matrix inversions are in the $\mathcal{I}_k^-$ and K_k equations. These inversions are scalar in this example because there is only one state in this example.

▽▽▽

6.3 SQUARE ROOT FILTERING

The early days of Kalman filtering in the 1960s saw a lot of promise and successful applications in the aerospace industry and in NASA's space program, but sometimes problems arose in implementation. Many of the problems that were encountered were due to numerical difficulties. The Riccati equation solution P_k should theoretically always be a symmetric positive semidefinite matrix, but numerical problems in computer implementations sometimes led to P_k matrices that became indefinite or nonsymmetric. This was often because of the short word lengths in the computers of the 1960s [Sch81]. Numerical problems may arise in cases in which some elements of the state-vector x are estimated to much greater precision than other elements of x . This could be because of discrepancies in the units of the state-vector elements. For example, one state might be in units of miles and can be estimated to within 0.01 miles, whereas a second state might be in units of cm/s and can be estimated to within 10 cm/s. The covariance for the first state would be on the order of 10^{-4} , whereas the covariance for the second state would be on the order of 10^2 . This led to a lot of research during the 1960s that was related to numerical implementations.

Square root filtering is a way to mathematically increase the precision of the Kalman filter when hardware precision is not available. Perhaps the first square root algorithm was developed by James Potter for NASA's Apollo space program [Bat64]. Although Potter's algorithm was limited to zero process noise and scalar measurements, its success led to a lot of additional square root research in the following years. Potter's algorithm was extended to handle process noise in [And68, Dye69], and was generalized in two different ways to handle vector measurements in [Bel67, And68]. Paul Kaminski gives a good review of square root filtering developments during the first decade of the Kalman filter [Kam71].

Now that computers have become so much more capable, we do not have to worry about numerical problems as often. Nevertheless, numerical issues still arise in finite-word-length implementations of algorithms, especially in embedded systems. In this section, we will discuss the square root filter, which was developed in order to

effectively increase the numerical precision of the Kalman filter and hence mitigate numerical difficulties in implementations. However, this improved performance is at the cost of greater computational effort. First, we will review the concept of the condition number of a matrix, then we will derive the square root version of the time update equation, and finally we will derive the square root version of the measurement update equations. Section 8.3.3 contains a discussion of square root filtering for the continuous-time Kalman filter.

6.3.1 Condition number

Recall the definition of the singular values of a matrix. An $n \times n$ matrix P has n singular values σ , given as

$$\begin{aligned}\sigma^2(P) &= \lambda(P^T P) \\ &= \lambda(P P^T)\end{aligned}\tag{6.36}$$

The matrix $P^T P$ is symmetric, and the eigenvalues of a symmetric matrix are always real and nonnegative, so the singular values of a matrix are always real and nonnegative. The matrix P is nonsingular (invertible) if and only if all of its singular values are positive. The condition number of a matrix is defined as

$$\begin{aligned}\kappa(P) &= \frac{\sigma_{\max}(P)}{\sigma_{\min}(P)} \\ &\geq 1\end{aligned}\tag{6.37}$$

Note that some authors use alternate definitions for condition number; for example, some authors define the condition number of a matrix as the square of the above definition.¹ As $\kappa(P) \rightarrow \infty$, the matrix P is said to be poorly conditioned or ill conditioned, and P approaches a singular matrix. In the implementation of a Kalman filter, the error covariance matrix P should always be positive definite because $P = E[(x - \hat{x})(x - \hat{x})^T]$. We use the standard notation

$$P > 0\tag{6.38}$$

to indicate that P is positive definite. This is equivalent to saying that P is invertible, which is equivalent to saying that all of the eigenvalues of P are greater than zero. But suppose in our Kalman filter that some elements of x are estimated to much greater precision than other elements of x . For example, suppose that

$$P = \begin{bmatrix} 10^6 & 0 \\ 0 & 10^{-6} \end{bmatrix}\tag{6.39}$$

This means that our estimate of x_1 has a standard deviation of 10^3 and our estimate of x_2 has a standard deviation of 10^{-3} . This could be due to drastically different units in x_1 and x_2 , or it could be simply that x_1 is much more observable than x_2 . The singular values of a diagonal matrix are the magnitudes of the diagonal elements, which are 10^6 and 10^{-6} . In other words,

$$\kappa(P) = 10^{12}\tag{6.40}$$

¹In MATLAB the COND function can be used to find the condition number of a matrix.

This is a pretty large condition number, which means that the P matrix might look like a singular matrix to a digital computer. For example, if we have a fixed-point computer with 10 decimal digits of precision and the 10^6 term is represented correctly in the computer, then the 10^{-6} term will be represented as a zero in the computer. Mathematically, P is nonsingular, but computationally P is singular.

The square root filter is based on the idea of finding an S matrix such that $P = SS^T$. The S matrix is then called a square root of P . Note that the definition of the square root of P is *not* that $P = S^2$, but that $P = SS^T$. Also note that this definition of the matrix square root is not standard. Some books and papers defined the matrix square root as $P = S^2$, others define it as $P = S^T S$, and others define it as $P = SS^T$. This latter definition is the one that we will use in this book. If P is symmetric positive definite then it always has a square root [Gol89, Moo00]. The square root of a matrix may not be unique; that is, there may be more than one solution for S in the equation $P = SS^T$. (This is analogous to the scalar square root, which is usually not unique. For example, the number 1 has two square roots; +1 and -1.) Also note that SS^T will always be symmetric positive semidefinite no matter what the value of the S matrix. Whereas numerical difficulties might cause P to become nonsymmetric or indefinite in the Kalman filter equations, numerical difficulties can never cause SS^T to become nonsymmetric or indefinite.

Matrix square root algorithms were first given by the French military officer Andre Cholesky (1875-1918) and the Polish astronomer Tadeusz Banachiewicz (1882-1954) [Fad59]. An interesting biography of Cholesky is given in the appendix of [Mai84].

The following algorithm computes an S matrix such that $P = SS^T$ for an $n \times n$ matrix P .

The Cholesky Matrix Square Root Algorithm {

For $i = 1, \dots, n$

{

$$S_{ii} = \sqrt{P_{ii} - \sum_{j=1}^{i-1} S_{ij}^2}$$

For $j = 1, \dots, n$

{

$$S_{ji} = 0 \quad j < i$$

$$S_{ji} = \frac{1}{S_{ii}} \left(P_{ji} - \sum_{k=1}^{i-1} S_{jk} S_{ik} \right) \quad j > i$$

}

}

}

This is called Cholesky factorization and results in a matrix S such that $P = SS^T$. The matrix S is referred to as the Cholesky triangle because it is a lower triangular matrix. However, the algorithm only works if P is symmetric positive definite. If P is not symmetric positive definite, then it may or may not have a square root.²

In the following example we illustrate the application of Cholesky factorization.

²The MATLAB function CHOL outputs the transpose of the Cholesky triangle that is computed above.

■ EXAMPLE 6.3

This example is taken from [Kam71]. Suppose we have a P matrix given as

$$P = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 8 & 2 \\ 3 & 2 & 14 \end{bmatrix} \quad (6.41)$$

The Cholesky factorization algorithm tells us that, for $i = 1$,

$$\begin{aligned} S_{11} &= \sqrt{P_{11}} \\ &= 1 \\ S_{21} &= \frac{1}{S_{11}} (P_{21}) \\ &= 2 \\ S_{31} &= \frac{1}{S_{11}} (P_{31}) \\ &= 3 \end{aligned} \quad (6.42)$$

For $i = 2$, the algorithm tells us that

$$\begin{aligned} S_{22} &= \sqrt{P_{22} - \sum_{j=1}^1 S_{2j}^2} \\ &= 2 \\ S_{12} &= 0 \\ S_{32} &= \frac{1}{S_{22}} \left(P_{32} - \sum_{k=1}^1 S_{3k} S_{2k} \right) \\ &= -2 \end{aligned} \quad (6.43)$$

For $i = 3$, the algorithm tells us that

$$\begin{aligned} S_{33} &= \sqrt{P_{33} - \sum_{j=1}^2 S_{3j}^2} \\ &= 1 \\ S_{13} &= 0 \\ S_{23} &= 0 \end{aligned} \quad (6.44)$$

So we obtain

$$S = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 2 & 0 \\ 3 & -2 & 1 \end{bmatrix} \quad (6.45)$$

and it can be verified that $P = SS^T$.

▽▽▽

After defining S as the square root of P in the Kalman filter, we will propagate S instead of P . This requires more computational effort but it doubles the precision

of the filter and helps prevent numerical problems. The singular values σ of P are given as

$$\begin{aligned}\sigma^2(P) &= \lambda(P^T P) \\ &= \lambda(SS^T SS^T)\end{aligned}\quad (6.46)$$

The singular values of S are given as

$$\sigma^2(S) = \lambda(SS^T) \quad (6.47)$$

Recall that for a general matrix A we have $\lambda(A^2) = \lambda^2(A)$. Therefore, we see from the above equations that

$$\begin{aligned}\sigma^2(P) &= [\sigma^2(S)]^2 \\ \frac{\sigma_{\max}(P)}{\sigma_{\min}(P)} &= \frac{\sigma_{\max}^2(S)}{\sigma_{\min}^2(S)} \\ \kappa(P) &= \kappa^2(S)\end{aligned}\quad (6.48)$$

That is, the condition number of P is the square of the condition number of S . For example, consider the P matrix given earlier in this section:

$$\begin{aligned}P &= \begin{bmatrix} 10^6 & 0 \\ 0 & 10^{-6} \end{bmatrix} \\ \kappa(P) &= 10^{12}\end{aligned}\quad (6.49)$$

The square root of this matrix and its condition number are

$$\begin{aligned}S &= \begin{bmatrix} 10^3 & 0 \\ 0 & 10^{-3} \end{bmatrix} \\ \kappa(S) &= 10^6\end{aligned}\quad (6.50)$$

The condition number of P is 10^{12} , but the condition number of the square root of P is only 10^6 . Square root filtering uses this idea to provide twice the precision of the standard Kalman filter. Instead of propagating P , we propagate the square root of P .

6.3.2 The square root time-update equation

Suppose we have an n -state discrete LTI system given as

$$\begin{aligned}x_k &= F_{k-1}x_{k-1} + G_{k-1}u_{k-1} + w_{k-1} \\ E(w_k w_k^T) &= Q_k\end{aligned}\quad (6.51)$$

The *a priori* error covariance matrix of the Kalman filter is P_k^- , and its square root is S_k^- . The *a posteriori* error covariance matrix is P_k^+ , and its square root is S_k^+ . Suppose that we can find an orthogonal $2n \times 2n$ matrix T such that

$$\begin{aligned}\begin{bmatrix} (S_k^-)^T \\ 0 \end{bmatrix} &= T \begin{bmatrix} (S_{k-1}^+)^T F_{k-1}^T \\ Q_{k-1}^{T/2} \end{bmatrix} \\ &= \begin{bmatrix} T_1 & T_2 \end{bmatrix} \begin{bmatrix} (S_{k-1}^+)^T F_{k-1}^T \\ Q_{k-1}^{T/2} \end{bmatrix}\end{aligned}\quad (6.52)$$

Since T is orthogonal we see that

$$\begin{aligned}
 T^T T &= \begin{bmatrix} T_1^T \\ T_2^T \end{bmatrix} \begin{bmatrix} T_1 & T_2 \end{bmatrix} \\
 &= \begin{bmatrix} T_1^T T_1 & T_1^T T_2 \\ T_2^T T_1 & T_2^T T_2 \end{bmatrix} \\
 &= \begin{bmatrix} I & 0 \\ 0 & I \end{bmatrix}
 \end{aligned} \tag{6.53}$$

where T_1 and T_2 are both $n \times n$ matrices. We see from the above that

$$\begin{aligned}
 T_1^T T_2 &= T_2^T T_1 = 0 \\
 T_1^T T_1 &= T_2^T T_2 = I
 \end{aligned} \tag{6.54}$$

Now note that we can use Equation (6.52) to write

$$\begin{bmatrix} S_k^- & 0 \end{bmatrix} \begin{bmatrix} (S_k^-)^T \\ 0 \end{bmatrix} = \begin{bmatrix} T_1 (S_{k-1}^+)^T F_{k-1}^T + T_2 Q_{k-1}^{T/2} \end{bmatrix}^T \begin{bmatrix} \dots \end{bmatrix} \tag{6.55}$$

We can use this equation, along with Equation (6.54), to write

$$\begin{aligned}
 S_k^- (S_k^-)^T &= F_{k-1} S_{k-1}^+ T_1^T T_1 (S_{k-1}^+)^T F_{k-1}^T + Q_{k-1}^{1/2} T_2^T T_2 Q_{k-1}^{T/2} \\
 &= F_{k-1} S_{k-1}^+ (S_{k-1}^+)^T F_{k-1}^T + Q_{k-1}^{1/2} Q_{k-1}^{T/2}
 \end{aligned} \tag{6.56}$$

If S_{k-1}^+ is the square root of P_{k-1}^+ , this implies that

$$P_k^- = F_{k-1} P_{k-1}^+ F_{k-1}^T + Q_{k-1} \tag{6.57}$$

which is exactly the time-update equation for P_k that is required in the Kalman filter, as shown in Equation (5.19). So if we can find an orthogonal $2n \times 2n$ matrix T such that

$$T \begin{bmatrix} (S_{k-1}^+)^T F_{k-1}^T \\ Q_{k-1}^{T/2} \end{bmatrix} = \begin{bmatrix} n \times n \text{ matrix} \\ 0 \end{bmatrix} \tag{6.58}$$

then the $n \times n$ matrix in the upper half of the matrix on the right side is equal to $(S_k^-)^T$. This assumes that $(S_{k-1}^+)^T$ is available from a square root measurement update equation, which we will discuss in the following two subsections. The square root time update equation above is mathematically equivalent to the original Kalman filter time update equation for P , but the update equation is used to update S instead of P .

As we noted above, the square root of P_k^- is not unique, so different algorithms for solving Equation (6.58) will result in different T and $(S_k^-)^T$ matrices. We can use various methods from numerical linear algebra to find the orthogonal $2n \times 2n$ matrix T and the resulting square root matrix S_k^- (e.g., Householder, Gram-Schmidt, modified Gram-Schmidt, or Givens transformations) [Hor85, Gol89, Str90, Moo00]. A couple of these algorithms are discussed in Section 6.3.5.

■ EXAMPLE 6.4

Suppose that at time $(k-1)$ our Kalman filter has a system matrix, process noise covariance, and a *posteriori* estimation covariance square root equal to

$$\begin{aligned} F_{k-1} &= \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \\ Q_{k-1} &= \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix} \\ S_{k-1}^+ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned} \quad (6.59)$$

It can be verified that the square root of Q_{k-1} (so that $Q_{k-1}^{1/2} Q_{k-1}^{T/2} = Q_{k-1}$) is given by

$$Q_{k-1}^{1/2} = \begin{bmatrix} 0 & 0 \\ -1 & -1 \end{bmatrix} \quad (6.60)$$

Equation (6.58) can be solved as

$$\begin{aligned} T \begin{bmatrix} (S_{k-1}^+)^T F_{k-1}^T \\ Q_{k-1}^{T/2} \end{bmatrix} &= \begin{bmatrix} (S_k^-)^T \\ 0 \end{bmatrix} \\ \frac{1}{\sqrt{10}} \begin{bmatrix} \sqrt{5} & \sqrt{5} & 0 & 0 \\ 1 & -1 & 2 & 2 \\ -2 & -2 & 1 & 1 \\ 0 & 0 & -\sqrt{5} & \sqrt{5} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & -1 \\ 0 & -1 \end{bmatrix} &= \frac{1}{\sqrt{10}} \begin{bmatrix} \sqrt{20} & \sqrt{5} \\ 0 & -5 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \end{aligned} \quad (6.61)$$

As mentioned earlier, algorithms for performing this computation will be discussed in Section 6.3.5. The upper-right square matrix on the right side of the above equation is equal to $(S_k^-)^T$, so this shows that the square root of the *a priori* estimation covariance at time k is given as

$$S_k^- = \frac{1}{\sqrt{10}} \begin{bmatrix} \sqrt{20} & 0 \\ \sqrt{5} & -5 \end{bmatrix} \quad (6.62)$$

From this it can be inferred that the *a priori* estimation covariance at time k is given as

$$\begin{aligned} P_k^- &= S_k^- (S_k^-)^T \\ &= \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \end{aligned} \quad (6.63)$$

Indeed, a straightforward implementation of the time-update equation for the estimation-error covariance gives

$$\begin{aligned} P_k^- &= F_{k-1} P_{k-1}^+ F_{k-1}^T + Q_{k-1} \\ &= \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \end{aligned} \quad (6.64)$$

which confirms our square root results. However, the square root time update has essentially twice the precision of the standard time-update equation.

▽▽▽

6.3.3 Potter's square root measurement-update equation

The square root measurement-update equation discussed here is based on James Potter's algorithm, which was developed for NASA's Apollo space program [Bat64, Kam71] and modified by Angus Andrews to handle vector measurements [And68]. Recall from Equation (5.19) that the measurement update equation for the estimation covariance is given as

$$P_k^+ = (I - K_k H_k) P_k^- \quad (6.65)$$

We can process the measurements one at a time using the sequential Kalman filter of Section 6.1. That is, first we initialize $P_{0k}^+ = P_k^-$. Then, for $i = 1, \dots, r$ (where r is the number of measurements), we compute

$$\begin{aligned} K_{ik} &= \frac{P_{i-1,k}^+ H_{ik}^T}{H_{ik} P_{i-1,k}^+ H_{ik}^T + R_{ik}} \\ P_{ik}^+ &= (I - K_{ik} H_{ik}) P_{i-1,k}^+ \end{aligned} \quad (6.66)$$

where H_{ik} is the i th row of H_k and R_{ik} is the variance of the i th measurement. (We are assuming here, as in Section 6.1, that R_k is diagonal.) Suppose we have the square root of $P_{i-1,k}^+$ so that $P_{i-1,k}^+ = S_{i-1,k}^+ S_{i-1,k}^{+T}$. Then K_{ik} can be written as

$$K_{ik} = \frac{S_{i-1,k}^+ S_{i-1,k}^{+T} H_{ik}^T}{H_{ik} S_{i-1,k}^+ S_{i-1,k}^{+T} H_{ik}^T + R_{ik}} \quad (6.67)$$

and P_{ik}^+ can be written as

$$\begin{aligned} P_{ik}^+ &= \left(I - \frac{S_{i-1,k}^+ S_{i-1,k}^{+T} H_{ik}^T H_{ik}}{H_{ik} S_{i-1,k}^+ S_{i-1,k}^{+T} H_{ik}^T + R_{ik}} \right) S_{i-1,k}^+ S_{i-1,k}^{+T} \\ &= S_{i-1,k}^+ (I - a \phi \phi^T) S_{i-1,k}^{+T} \end{aligned} \quad (6.68)$$

where ϕ and a are defined as

$$\begin{aligned} \phi &= S_{i-1,k}^+ H_{ik}^T \\ a &= \frac{1}{\phi^T \phi + R_{ik}} \end{aligned} \quad (6.69)$$

It can be shown (see Problem 6.9) that

$$I - a \phi \phi^T = (I - a \gamma \phi \phi^T)^2 \quad (6.70)$$

where γ is given as

$$\gamma = \frac{1}{1 \pm \sqrt{a R_{ik}}} \quad (6.71)$$

Either the plus or minus sign can be used in the computation of γ . Comparing Equations (6.68) and (6.70) shows that

$$S_{ik}^+ = S_{i-1,k}^+ (I - a \gamma \phi \phi^T) \quad (6.72)$$

This results in a square root measurement-update algorithm that can be summarized as follows.

Potter's square root measurement-update algorithm

1. After the *a priori* covariance square root S_k^- and the *a priori* state estimate $\hat{x}_k^-$ have been computed, initialize

$$\begin{aligned}\hat{x}_{0k}^+ &= \hat{x}_k^- \\ S_{0k}^+ &= S_k^-\end{aligned}\tag{6.73}$$

2. For $i = 1, \dots, r$ (where r is the number of measurements), perform the following.
 - (a) Define H_{ik} as the i th row of H_k , y_{ik} as the i th element of y_k , and R_{ik} as the variance of the i th measurement (assuming that R_k is diagonal).
 - (b) Perform the following to find the square root of the covariance after the i th measurement has been processed:

$$\begin{aligned}\phi_i &= S_{i-1,k}^{+T} H_{ik}^T \\ a_i &= \frac{1}{\phi_i^T \phi_i + R_{ik}} \\ \gamma_i &= \frac{1}{1 \pm \sqrt{a_i R_{ik}}} \\ S_{ik}^+ &= S_{i-1,k}^+ (I - a_i \gamma_i \phi_i \phi_i^T)\end{aligned}\tag{6.74}$$

- (c) Compute the Kalman gain for the i th measurement as

$$K_{ik} = a_i S_{ik}^+ \phi_i \tag{6.75}$$

- (d) Compute the state estimate update due to the i th measurement as

$$\hat{x}_{ik}^+ = \hat{x}_{i-1,k}^+ + K_{ik} (y_{ik} - H_{ik} \hat{x}_{i-1,k}^+) \tag{6.76}$$

3. Set the *a posteriori* covariance square root and the *a posteriori* state estimate as

$$\begin{aligned}S_k^+ &= S_{rk}^+ \\ \hat{x}_k^+ &= \hat{x}_{rk}^+\end{aligned}\tag{6.77}$$

Although square root filtering improves the numerical characteristics of the Kalman filter, it also increases computational requirements. Efforts to make square root filtering more efficient are reported in [Car73, Tho77, Tap80].

EXAMPLE 6.5

This example is based on [Kam71]. Suppose that we have an LTI system with

$$\begin{aligned}P_k^- &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ H &= \begin{bmatrix} 1 & 0 \end{bmatrix} \\ F &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ Q &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}\end{aligned}\tag{6.78}$$

If we had an infinite-precision computer, the exact Kalman gain and *a posteriori* covariance at time k would be given by

$$\begin{aligned} K_k &= P_k^- H^T (H P_k^- H^T + R)^{-1} \\ &= \begin{bmatrix} \frac{1}{1+R} \\ 0 \end{bmatrix} \\ P_k^+ &= (I - K_k H) P_k^- \\ &= \begin{bmatrix} \frac{R}{1+R} & 0 \\ 0 & 1 \end{bmatrix} \end{aligned} \quad (6.79)$$

The *a priori* covariance and Kalman gain at the next time step $(k+1)$ would be given by

$$\begin{aligned} P_{k+1}^- &= F P_k^+ F^T + Q \\ &= \begin{bmatrix} \frac{R}{1+R} & 0 \\ 0 & 1 \end{bmatrix} \\ K_{k+1} &= P_{k+1}^- H^T (H P_{k+1}^- H^T + R)^{-1} \\ &= \begin{bmatrix} \frac{1}{2+R} \\ 0 \end{bmatrix} \end{aligned} \quad (6.80)$$

Now consider implementation in a finite precision digital computer. Suppose that the measurement covariance $R \ll 1$. The covariance R is such a tiny number that because of rounding in the computer, $1+R = 1$, but $1+\sqrt{R} > 1$. The rounded values of the Kalman gain and *a posteriori* covariance at time k would be given by

$$\begin{aligned} K_k &= \begin{bmatrix} \frac{1}{1+R} \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ P_k^+ &= (I - K_k H) P_k^- \\ &= \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned} \quad (6.81)$$

Note that P_k^+ has become singular because of the numerical limitations of the computer. The rounded values of the *a priori* covariance and Kalman gain at the next time step $(k+1)$ would be given by

$$\begin{aligned} P_{k+1}^- &= F P_k^+ F^T + Q \\ &= \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \\ K_{k+1} &= P_{k+1}^- H^T (H P_{k+1}^- H^T + R)^{-1} \\ &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} \end{aligned} \quad (6.82)$$

The numerical limitations of the computer have resulted in a zero Kalman gain, whereas the infinite-precision Kalman gain as given in Equation (6.80) is about $\begin{bmatrix} 1/2 & 0 \end{bmatrix}^T$.

Now suppose we implement the measurement-update equation using Potter's algorithm. We start out with

$$S_k^- = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (6.83)$$

We only have to iterate through Equation (6.74) one time since we only have one measurement. The rounded values of the parameters given in Equation (6.74) are

$$\begin{aligned} \phi &= (S_k^-)^T H^T \\ &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ a &= \frac{1}{\phi^T \phi + R} \\ &= \frac{1}{1 + R} \\ &= 1 \\ \gamma &= \frac{1}{1 + \sqrt{aR}} \\ &= \frac{1}{1 + \sqrt{R}} \\ S_k^+ &= S_k^- (I - a\gamma\phi\phi^T) \\ &= \begin{bmatrix} \frac{\sqrt{R}}{1+\sqrt{R}} & 0 \\ 0 & 1 \end{bmatrix} \end{aligned} \quad (6.84)$$

Note that $S_k^+ S_k^{+T}$ is nonsingular. The rounded values of the square root of the *a priori* covariance, the parameters of Equation (6.74), and the Kalman gain at the next time step ($k+1$) would be given by

$$\begin{aligned} S_{k+1}^- &= S_k^+ \\ \phi &= (S_{k+1}^-)^T H^T \\ &= \begin{bmatrix} \frac{\sqrt{R}}{1+\sqrt{R}} \\ 0 \end{bmatrix} \\ a &= \frac{1}{\phi^T \phi + R} \\ &= \frac{1 + R + 2\sqrt{R}}{R^2 + 2R + 2R\sqrt{R}} \\ &= \frac{1 + 2\sqrt{R}}{2R + 2R\sqrt{R}} \\ K_{k+1} &= aS_{k+1}^- \phi \\ &= \frac{1 + 2\sqrt{R}}{2R(1 + \sqrt{R})} \begin{bmatrix} \frac{R}{1+R+2\sqrt{R}} \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} \frac{1}{2(1+\sqrt{R})} \\ 0 \end{bmatrix} \end{aligned} \quad (6.85)$$

Note that the rounded Kalman gain is almost identical to the exact Kalman gain given by Equation (6.80). This shows the benefit that can be gained by using the square root filter.

▽▽▽

6.3.4 Square root measurement update via triangularization

The previous section derived a measurement update based on Potter's algorithm that could be performed on the square root of the Kalman filter estimation covariance. This section derives an alternative method for performing the measurement update. Suppose that we want to design a Kalman filter for a system with n states and r measurements. Suppose that we can find an orthogonal matrix $(n+r) \times (n+r)$ matrix $\tilde{T}$ such that

$$\begin{bmatrix} (R_k + H_k P_k^- H_k^T)^{T/2} & \tilde{K}_k^T \\ 0 & (S_k^+)^T \end{bmatrix} = \tilde{T} \begin{bmatrix} R_k^{T/2} & 0 \\ (S_k^-)^T H_k^T & (S_k^-)^T \end{bmatrix} \quad (6.86)$$

S_k^- and S_k^+ are the square roots of the *a priori* and *a posteriori* covariances, and $\tilde{K}_k$ is defined as

$$\tilde{K}_k = K_k (R_k + H_k P_k^- H_k^T)^{T/2} \quad (6.87)$$

where K_k is the normal Kalman gain matrix. Note that S_k^+ in Equation (6.86) is not known until after an orthogonal $\tilde{T}$ is found that forces the left side of Equation (6.86) into the specified form. That is, we need to find a $\tilde{T}$ so that the upper-left $r \times r$ block of the left side of Equation (6.86) is equal to $(R_k + H_k P_k^- H_k^T)^{T/2}$, the upper-right $r \times n$ block is equal to $\tilde{K}_k^T$, and the lower-left $n \times r$ block is equal to 0. After such a $\tilde{T}$ is found, whatever the lower right $n \times n$ block turns out to be is, by definition, equal to $(S_k^+)^T$, which is the transpose of the square root of P_k^+ . Now write the $(n+r) \times (n+r)$ matrix $\tilde{T}$ as

$$\tilde{T} = \begin{bmatrix} \tilde{T}_{11} & \tilde{T}_{12} \\ \tilde{T}_{21} & \tilde{T}_{22} \end{bmatrix} \quad (6.88)$$

where $\tilde{T}_{11}$ is an $r \times r$ matrix, $\tilde{T}_{12}$ is an $r \times n$ matrix, $\tilde{T}_{21}$ is an $n \times r$ matrix, and $\tilde{T}_{22}$ is an $n \times n$ matrix. Since $\tilde{T}$ is orthogonal we can write

$$\begin{aligned} \tilde{T}^T \tilde{T} &= \begin{bmatrix} \tilde{T}_{11}^T & \tilde{T}_{21}^T \\ \tilde{T}_{12}^T & \tilde{T}_{22}^T \end{bmatrix} \begin{bmatrix} \tilde{T}_{11} & \tilde{T}_{12} \\ \tilde{T}_{21} & \tilde{T}_{22} \end{bmatrix} \\ &= \begin{bmatrix} \tilde{T}_{11}^T \tilde{T}_{11} + \tilde{T}_{21}^T \tilde{T}_{21} & \tilde{T}_{11}^T \tilde{T}_{12} + \tilde{T}_{21}^T \tilde{T}_{22} \\ \tilde{T}_{12}^T \tilde{T}_{11} + \tilde{T}_{22}^T \tilde{T}_{21} & \tilde{T}_{12}^T \tilde{T}_{12} + \tilde{T}_{22}^T \tilde{T}_{22} \end{bmatrix} \\ &= \begin{bmatrix} I & 0 \\ 0 & I \end{bmatrix} \end{aligned} \quad (6.89)$$

Now we expand Equation (6.86) as

$$\begin{bmatrix} (R_k + H_k P_k^- H_k^T)^{T/2} & \tilde{K}_k^T \\ 0 & (S_k^+)^T \end{bmatrix} = \begin{bmatrix} \tilde{T}_{11} R_k^{T/2} + \tilde{T}_{12} (S_k^-)^T H_k^T & \tilde{T}_{12} (S_k^-)^T \\ \tilde{T}_{21} R_k^{T/2} + \tilde{T}_{22} (S_k^-)^T H_k^T & \tilde{T}_{22} (S_k^-)^T \end{bmatrix} \quad (6.90)$$

We will equate the four matrix partitions of this equation to write four separate equalities. We will then take each equality and premultiply each side by its transpose to obtain four new equalities. The first two equalities obtained this way are

$$\begin{aligned}
 (R_k + H_k P_k^- H_k^T)^{1/2} (\dots)^{T/2} = & R_k^{1/2} \tilde{T}_{11}^T \tilde{T}_{11} R_k^{T/2} + H_k S_k^- \tilde{T}_{12}^T \tilde{T}_{11} R_k^{T/2} + \\
 & R_k^{1/2} \tilde{T}_{11}^T \tilde{T}_{12} (S_k^-)^T H_k^T + H_k S_k^- \tilde{T}_{12}^T \tilde{T}_{12} (S_k^-)^T H_k^T \\
 0 = & R_k^{1/2} \tilde{T}_{21}^T \tilde{T}_{21} R_k^{T/2} + R_k^{1/2} \tilde{T}_{21}^T \tilde{T}_{22} (S_k^-)^T H_k^T + \\
 & H_k S_k^- \tilde{T}_{22}^T \tilde{T}_{21} R_k^{T/2} + H_k S_k^- \tilde{T}_{22}^T \tilde{T}_{22} (S_k^-)^T H_k^T
 \end{aligned} \quad (6.91)$$

Adding these two equations and using Equations (6.87) and (6.89) to simplify the result gives

$$R_k + H_k P_k^- H_k^T = R_k + H_k S_k^- (S_k^-)^T H_k^T \quad (6.92)$$

This shows that the proposed measurement update of Equation (6.86) is consistent with S_k^- being the square root of P_k^- .

The second two equalities that can be written from Equation (6.90) are

$$\begin{aligned}
 \tilde{K}_k \tilde{K}_k^T &= S_k^- \tilde{T}_{12}^T \tilde{T}_{12} (S_k^-)^T \\
 S_k^+ (S_k^+)^T &= S_k^- \tilde{T}_{22}^T \tilde{T}_{22} (S_k^-)^T
 \end{aligned} \quad (6.93)$$

Adding these two equations and using Equation (6.89) to simplify the result gives

$$S_k^+ (S_k^+)^T + K_k (R_k + H_k P_k^- H_k^T) K_k^T = S_k^- (S_k^-)^T \quad (6.94)$$

Substituting the standard Kalman gain equation $K_k = P_k^- H_k^T (R_k + H_k P_k^- H_k^T)^{-1}$ into this equation gives

$$\begin{aligned}
 S_k^+ (S_k^+)^T + P_k^- H_k^T K_k^T &= P_k^- \\
 S_k^+ (S_k^+)^T &= P_k^- - P_k^- H_k^T K_k^T
 \end{aligned} \quad (6.95)$$

Since the left side of the above equation is symmetric and the first term on the right side is symmetric, the last term on the right side must also be symmetric, which means that we can transpose it in the above equation to obtain

$$S_k^+ (S_k^+)^T = P_k^- - K_k H_k P_k^- \quad (6.96)$$

The right side of this equation is the Kalman filter measurement-update equation for P , which means that the left side of the equation must be P_k^+ , which means that S_k^+ must be the square root of P_k^+ . So if we can find an orthogonal $(n+r) \times (n+r)$ matrix $\tilde{T}$ such that

$$\begin{bmatrix} (R_k + H_k P_k^- H_k^T)^{T/2} & \tilde{K}_k^T \\ 0 & (n \times n \text{ matrix}) \end{bmatrix} = \tilde{T} \begin{bmatrix} R_k^{T/2} & 0 \\ (S_k^-)^T H_k^T & (S_k^-)^T \end{bmatrix} \quad (6.97)$$

then the lower-right $n \times n$ matrix on the left side of the equation is equal to the transpose of the square root of P_k^+ , and this equation is mathematically equivalent to the original Kalman filter measurement-update equation for P_k . This measurement-update method results in numerical precision that is effectively twice as much as the standard Kalman filter, which helps to avoid numerical problems. However, the computation of $\tilde{T}$ adds a lot of computational effort to the Kalman filter. In addition, the form of the transformation given in Equation (6.97) makes it of questionable practicality (see Problem 6.10).

6.3.5 Algorithms for orthogonal transformations

Several numerical algorithms are available for performing the orthogonal transformations that are required to solve for the T and S_k^- matrices in Equation (6.58). Some algorithms that can be used are the Householder method, the Givens method, the Gram-Schmidt method, and the modified Gram-Schmidt method. In this section we will present (without derivation) the Householder algorithm and the modified Gram-Schmidt algorithm. Derivations and presentations of the other algorithms can be found in many texts on numerical linear algebra, such as [Hor85, Gol89, Moo00]. A comparison of Gram-Schmidt, modified Gram-Schmidt, and Householder transformations can be found in [Jor68], where it is stated that the modified Gram-Schmidt procedure is best (from a numerical point of view), with the Householder method offering competitive performance.

6.3.5.1 The Householder algorithm The algorithm presented here was developed by Alston Householder [Hou64, Chapter 5], applied to least squares estimation by Gene Golub [Gol65], and summarized for Kalman filtering by Paul Kaminiski [Kam71].

1. Suppose that we have a $2n \times n$ matrix $A^{(1)}$, and we want to find an $n \times n$ matrix W such that

$$TA^{(1)} = \begin{bmatrix} W \\ 0 \end{bmatrix} \quad (6.98)$$

where T is an orthogonal $2n \times 2n$ matrix, and 0 is the $n \times n$ matrix consisting of all zeros. Note that this problem statement is in the same form as Equation (6.58). Also note that we do not necessarily need to find T ; our goal is to find W .

2. For $k = 1, \dots, n$ perform the following:

- (a) Compute the scalar σ_k as

$$\sigma_k = \text{sgn} \left(A_{kk}^{(k)} \right) \sqrt{\sum_{i=k}^{2n} \left(A_{ik}^{(k)} \right)^2} \quad (6.99)$$

where $A_{ik}^{(k)}$ is the element in the i th row and k th column of $A^{(k)}$. The $\text{sgn}(\cdot)$ function is defined to be equal to $+1$ if its argument is greater than or equal to zero, and -1 if its argument is less than zero.

- (b) Compute the scalar β_k as

$$\beta_k = \frac{1}{\sigma_k \left(\sigma_k + A_{kk}^{(k)} \right)} \quad (6.100)$$

- (c) For $i = 1, \dots, 2n$ perform the following:

$$u_i^{(k)} = \begin{cases} 0 & i < k \\ \sigma_k + A_{kk}^{(k)} & i = k \\ A_{ik}^{(k)} & i > k \end{cases} \quad (6.101)$$

This gives a $2n$ -element column vector $u^{(k)}$.

- (d) For $i = 1, \dots, n$ perform the following:

$$y_i^{(k)} = \begin{cases} 0 & i < k \\ 1 & i = k \\ \beta_k u^{(k)T} A_i^{(k)} & i > k \end{cases} \quad (6.102)$$

where $A_i^{(k)}$ is the i th column of $A^{(k)}$. This gives an n -element column vector $y^{(k)}$.

- (e) Compute the $2n \times n$ matrix $A^{(k+1)}$ as

$$A^{(k+1)} = A^{(k)} - u^{(k)} y^{(k)T} \quad (6.103)$$

3. After the above steps have been executed, $A^{(n+1)}$ has the form

$$A^{(n+1)} = \begin{bmatrix} W \\ 0 \end{bmatrix} \quad (6.104)$$

where W is the $n \times n$ matrix that we are trying to solve for. Note that if $\sigma_k = 0$ at any stage of the algorithm, that means $A^{(1)}$ is rank deficient and the algorithm will fail. Also note that the above algorithm does not compute the T matrix. However, we can find the T matrix as

$$\begin{aligned} T &= T^{(n)} T^{(n-1)} \dots T^{(1)} \\ T^{(k)} &= I - \beta_k u^{(k)} u^{(k)T} \quad i = 1, \dots, n \end{aligned} \quad (6.105)$$

6.3.5.2 The modified Gram–Schmidt algorithm The modified Gram–Schmidt algorithm for orthonormalization that is presented here is discussed in most linear systems books [Kai80, Bay99, Che99]. It was first given in [Bjo67] and was summarized for Kalman filtering in [Kam71].

1. Suppose that we have a $2n \times n$ matrix $A^{(1)}$, and we want to find an $n \times n$ matrix W such that

$$TA^{(1)} = \begin{bmatrix} W \\ 0 \end{bmatrix} \quad (6.106)$$

where T is an orthogonal $2n \times 2n$ matrix, and 0 is the $n \times n$ matrix consisting of all zeros. Note that this problem statement is in the same form as Equation (6.58).

2. For $k = 1, \dots, n$ perform the following.

- (a) Compute the scalar σ_k as

$$\sigma_k = \sqrt{A_k^{(k)T} A_k^{(k)}} \quad (6.107)$$

where $A_i^{(k)}$ is the i th column of $A^{(k)}$.

- (b) Compute the k th row of W as

$$W_{kj} = \begin{cases} 0 & j = 1, \dots, k-1 \\ \sigma_k & j = k \\ A_k^{(k)T} A_j^{(k)} / \sigma_k & j = k+1, \dots, n \end{cases} \quad (6.108)$$

(c) Compute the k th row of T as

$$T_k = A_k^{(k)T} / \sigma_k \quad (6.109)$$

(d) If $(k < n)$, compute the last $(n - k)$ columns of $A^{(k+1)}$ as

$$A_j^{(k+1)} = A_j^{(k)} - W_{kj} A_k^{(k)} / \sigma_k \quad j = k + 1, \dots, n \quad (6.110)$$

Note that the first k columns of $A^{(k+1)}$ are not computed in this algorithm.

As with the Householder algorithm, if $\sigma_k = 0$ at any stage of the algorithm, that means $A^{(1)}$ is rank deficient and the algorithm fails. After this algorithm completes, we have the first n rows of T , and T is an $n \times 2n$ matrix. If we want to know the last n rows of T , we can compute them using a regular Gram-Schmidt algorithm as follows [Hor85, Gol89, Moo00].

1. Fill out the T matrix that was begun above by appending a $2n \times 2n$ identity matrix to the bottom of it. This ensures that the rows of T span the entire $2n$ -dimensional vector space:

$$T = \begin{bmatrix} T \\ I \end{bmatrix} \quad (6.111)$$

Note that this T is a $3n \times 2n$ matrix.

2. Now we perform a standard Gram-Schmidt orthonormalization procedure on the last $2n$ rows of T (with respect to the already obtained first n rows of T). For $k = n + 1, \dots, 3n$, compute the k th row of T as

$$\begin{aligned} T_k &= T_k - \sum_{i=1}^{k-1} (T_k T_i^T) T_i \\ T_k &= \frac{T_k}{\|T_k\|_2} \end{aligned} \quad (6.112)$$

If T_k is zero then that means that it is a linear combination of the previous rows of T . In that case, the division in the above equation will be a divide by zero, so instead T_k should be discarded. This discard will actually occur exactly n times so that this procedure will compute n additional rows of T and we will end up with an orthogonal $2n \times 2n$ matrix T .

The Gram-Schmidt algorithms are named after the Danish mathematician Jorgen Gram (1850-1916) and the German mathematician Erhard Schmidt (1876-1959). Schmidt received his doctorate in 1905 under David Hilbert's supervision, and in 1929 he was on the doctoral committee of Eberhard Hopf (see Section 3.4.4). However, the Gram-Schmidt algorithm was actually invented by Pierre Laplace (1749-1827).